Product Kernel Perspective Spaces: Predicting Benchmark Scores from Sparse, Unpaired Model Evaluations

Anonymous

Abstract

Evaluating language models is expensive, and in practice benchmark coverage is *doubly sparse and noisy*: a model is run on only a subset of tasks, within a task on only a subset of queries, and the resulting per-cell score is a noisy average over those few queries. Methods that complete the (model $\times$ task) score matrix—low-rank matrix completion, item-response theory—use only the *aggregate* score of each cell. We argue that the cheapest way to evaluate better is to use *item-level* information: not just a cell's score but its individual responses. We introduce the **Product Kernel Perspective Space (PKPS)**, which embeds each model from its responses through a product kernel $k_Q \cdot k_R$ that compares responses to *similar* queries (not only identical ones), recovering the Data Kernel Perspective Space as the bandwidth $\sigma \to 0$ and degrading gracefully when models answer different queries. PKPS supplies an item-level *embedding* signal; ensembling it with the available item-level *score* signal—a cell's own sample mean where it was measured, matrix completion where it was not—recovers the full score matrix from observations where each cell has any number of queries (efficient benchmarking and benchmark completion become the two ends of one problem). We characterize *when* the embedding helps: given the same preprocessing as matrix completion, an effective-rank analysis shows the embedding *matches* completion at every rank on low-rank matrices and *beats* it when the matrix is high-rank (as on MATH), so effective rank sets the size of its advantage; it also survives unpaired queries where DKPS and IRT collapse, and rescues the small-cohort regime where low-rank completion fails for lack of rows. Across three observation levers—task coverage, queries per cell (noise), and cohort size—the single embedding ensemble beats the best score-only predictor on a heterogeneous 18-task, 93-model suite spanning math, translation, medical QA and legal classification, by a margin that grows as evaluation gets sparser, noisier, or smaller-cohort. Item-level information pays wherever real evaluation is incomplete, which is everywhere that matters.

1 Introduction

Comprehensive evaluation of language models is a growing bottleneck: leaderboards track hundreds of models across dozens of benchmarks, yet no model is run on everything. The resulting (model $\times$ task) score matrix is *doubly sparse*. *Across tasks*, a model is evaluated on a subset of benchmarks (missing tasks). *Within a task*, evaluation may use only a subset of queries to save cost (missing queries). Predicting the missing entries—a model's score on a task it has not been (fully) run on—would let practitioners reuse cached evaluations instead of paying to re-run them [Polo et al., 2024, Vivek et al., 2024, Papailiopoulos et al., 2026].

The Data Kernel Perspective Space (DKPS) [Anonymous, 2026] is a black-box approach: it embeds each model from its query responses, places models in a low-dimensional Euclidean space via classical multidimensional scaling, and regresses benchmark scores in that space. DKPS is query-efficient and competitive with item-response theory. But it has a structural limitation: the distance between two models is the average squared difference of their responses *to the same queries*, so DKPS requires a *paired* design. When models answer different queries—the common case in real, organically-assembled leaderboards—DKPS has nothing to compare and the information in those unpaired responses is thrown away.

We introduce the **Product Kernel Perspective Space (PKPS)**. The key change is to compare responses to *similar* queries, not only identical ones, by weighting each response-pair comparison with a query kernel. This (i) lets every unpaired response contribute, and (ii) contains DKPS as the special case where the query kernel is the identity. The broader thesis is that PKPS supplies an *item-level embedding* signal that score-completion baselines ignore, and that combining it with the available *item-level score* signal predicts benchmark performance more efficiently than either alone. Our contributions:

- **Method.** A product-kernel distance $k_Q \cdot k_R$ that generalizes DKPS, reduces to it as the query-kernel bandwidth $\sigma \to 0$, and whose bandwidth is chosen per run by a cheap median-scaled cross-validation (Section 3). We treat it as the embedding term of an embedding-$\oplus$-score ensemble.
- **Unified estimation problem.** We recover the full (model $\times$ task) matrix from observations where each cell has $m \geq 0$ queries, with one ensemble of the own sample (m>0), matrix completion (m=0), and the PKPS embedding (everywhere), structured by three levers—task coverage, queries per cell (noise), and cohort size (Section 4.1).
- **Where the embedding helps.** With the same preprocessing as matrix completion, an effective-rank analysis shows the embedding *matches* completion at every rank on low-rank matrices and *beats* it where the matrix is high-rank (MATH), so rank sets the size of its advantage; it survives unpaired queries where DKPS and IRT collapse; and it rescues the small-cohort regime where rank-2 completion fails for lack of rows (Section 4.2).
- **One ensemble, every lever.** On a heterogeneous 18-task, 93-model suite spanning math, translation, medical QA and legal classification, the embedding already beats matrix completion on its own across coverage and cohort, and the single ensemble beats the best score-only predictor across noise, coverage, and cohort—by a margin that grows as evaluation gets sparser, noisier, or smaller-cohort (Section 4.2).

2 Related work

Efficient benchmarking reduces evaluation cost by scoring on a subset of items: item-response theory [Polo et al., 2024, Rasch, 1960] and anchor points [Vivek et al., 2024] select informative items, while DKPS [Anonymous, 2026] predicts from cached responses in a perspective space. These assume a paired/anchored design. **Benchmark completion** treats the score matrix as low-rank and imputes missing entries [Papailiopoulos et al., 2026, Mazumder et al., 2010]; this uses only scores, not responses. PKPS unifies the two settings through a single product-kernel mechanism that consumes unpaired responses, and we show it is complementary to—and combinable with—the score-only low-rank view.

3 The Product Kernel Perspective Space

Models and observations. A *model* is a random mapping $f : \mathcal{Q} \to \mathcal{X}$ from a query space $\mathcal{Q}$ to a response space $\mathcal{X}$; querying f at q returns a response drawn from the distribution $f(q)$. We observe n models $f_1, \ldots, f_n$, each run on its own *query set* $Q_i \subseteq \mathcal{Q}$, giving a tuple $\mathbf{Q} = (Q_1, \ldots, Q_n)$ that may differ across models or be disjoint; the paired design $Q_1 = \cdots = Q_n$ is the special case. Two fixed, distinct embedding functions map raw objects to vectors: a *response* embedding $\text{embed}_R : \mathcal{X} \to \mathbb{R}^p$ and a *query* embedding $\text{embed}_Q : \mathcal{Q} \to \mathbb{R}^{p'}$ (e.g. different text-embedding models). For $q_j \in Q_i$ we average the embeddings of the r sampled responses, $x_{ij} = \frac{1}{r}\sum_{k=1}^{r} \text{embed}_R\big(f_i(q_j)^{(k)}\big) \in \mathbb{R}^p$, and write $u_j = \text{embed}_Q(q_j)$ for the embedded query. The goal is to predict each model's benchmark score $y_i \in \mathcal{Y}$.

Perspective space. Each method summarizes the n models in an $n \times n$ affinity matrix A (defined below). The squared distance between two models is $D^2(a,b) = A_{aa} + A_{bb} - 2A_{ab}$ (clamped at 0), and their d-dimensional *representations* are the classical-multidimensional-scaling minimizer

$$(\hat{\psi}_1, \ldots, \hat{\psi}_n) = \underset{z_1,\ldots,z_n \in \mathbb{R}^d}{\arg\min} \sum_{i,k=1}^{n} \big(\|z_i - z_k\| - D(i,k)\big)^2, \tag{1}$$

so model i is the point $\hat{\psi}_i \in \mathbb{R}^d$, its *perspective*. Paired DKPS and PKPS differ only in the affinity A.

Paired DKPS. With all n models answering the same m queries ($Q_1 = \cdots = Q_n$), DKPS uses the identity-matched affinity $A_{ab} = \frac{1}{m}\sum_j k_R(x_{aj}, x_{bj})$: response j of model a is paired only with response j of model b, so the design must be paired.

Product kernel. PKPS replaces identity matching with a query kernel k_Q:

$$A_{ab} = \frac{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l)\, k_R(x_{aj}, x_{bl})}{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l)}, \tag{2}$$

where Q_a is the query set of model a. Each term is normalized by its own query-kernel mass, so models that answered different numbers of (or entirely different) queries are still comparable; responses to *similar* queries—even across tasks—contribute and unpaired evaluations are used (Figure 1). Paired DKPS is the special case $k_Q = \delta$, where only $j = l$ survives.

Kernels. The response kernel k_R compares embedded responses, the query kernel k_Q compares embedded queries. Paired DKPS uses a linear response kernel and the identity query kernel $k_Q = \delta$. Throughout our analysis we use a linear or RBF k_R and an RBF query kernel over the embedded queries, $k_Q(q_j, q_l) = \exp(-\|u_j - u_l\|^2 / 2\sigma^2)$, with bandwidth σ chosen per run (below).

3.1 Theoretical properties of the PKPS

PKPS is a strict generalization of DKPS. **(i) DKPS limit.** Take $k_Q = \delta$, the indicator $\mathbf{1}[q_j = q_l]$. Since the double sum already ranges over $j \in Q_a$, $l \in Q_b$, a term survives only if the two queries are equal *and* that query lies in both sets, $\mathbf{1}[q_j = q_l]\,\mathbf{1}[q_j \in Q_a]\,\mathbf{1}[q_l \in Q_b] = \mathbf{1}[q_j = q_l \in Q_a \cap Q_b]$, so $A_{ab} = \frac{1}{|Q_a \cap Q_b|}\sum_{q \in Q_a \cap Q_b} k_R(x_{aq}, x_{bq})$ —exactly paired DKPS over the shared queries (the same-m-queries case recovers $\frac{1}{m}\sum_j k_R(x_{aj}, x_{bj})$). Moreover, as $\sigma \to 0$ the RBF kernel converges to δ (distinct queries have distinct embeddings), so DKPS is recovered in the small-bandwidth limit. **(ii) Unpaired comparability.** Because each affinity is normalized by its own query-kernel mass $\sum_{jl} k_Q(q_j, q_l)$, two models that answered different—or entirely disjoint—query sets remain comparable, whereas paired DKPS, which sums only over shared queries, is undefined when no query is shared. **(iii) Bandwidth interpolation.** Increasing σ smoothly trades exact query matching for cross-query bridging, so a single scalar controls how much unpaired evidence the perspective uses.

3.2 Inference in the PKPS

Inference is a two-step statistical problem, exactly as in the DKPS [Anonymous, 2026]: embed each model into the perspective space by the scaling of Eq. 1, then regress its benchmark score on the resulting coordinate. Concretely, on a set of reference models with known scores $\{(f_i, y_i)\}$ we fit a decision function $\hat{h} : \mathbb{R}^d \to \mathscr{Y}$ on the pairs $\{(\hat{\psi}_i, y_i)\}$ and predict any model's score as $\hat{y}(f) = \hat{h}(\hat{\psi}(f))$. The only free choices are the query-kernel bandwidth σ and the regressor $\hat{h}$, which we describe next; we then add the score channel that turns the embedding into an ensemble.

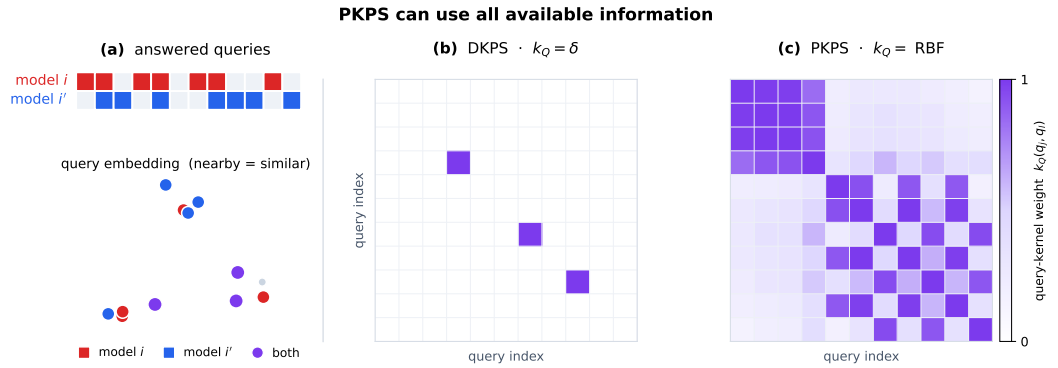

Figure 1: Why the query kernel matters under *unpaired* evaluation. **Left:** two models, i (red) and i' (blue), each answer their own subset of a shared query pool, shown both as answered-query blocks (top) and as points in query-embedding space (bottom; nearby points are similar queries). The two models share only a few queries. Both kernel panels index the queries by a single universal ordering (shared by rows and columns). **Center (DKPS, $k_Q = \delta$).** The identity query kernel keeps a cell only when the two queries are the *same* ($q_j = q_l$, the diagonal) *and* both models answered it ($q_j \in Q_i \cap Q_{i'}$), so only the few shared queries survive—almost every response is discarded. **Right (PKPS, $k_Q =$ RBF).** The RBF query kernel over the same ordering is a dense, *symmetric* similarity matrix, so responses to *similar* queries—not only identical ones—bridge the two models and every response can contribute. PKPS thus turns the sparse shared-query diagonal DKPS depends on into a dense similarity kernel.

Choosing the bandwidth. The optimal σ depends on how paired the data is: small when query overlap is high (recover exact matching), large when overlap is low (bridge across queries). We set it per run by cross-validation over a median-scaled grid $\sigma \in \{0.02, \ldots, 5\} \cdot \text{med}$, where med is the median pairwise query distance. The selection signal differs with the data: when dense reference scores are available, leave-one-family-out gives a low-noise criterion and the grid search is reliable; when only a sparse held-out set is available we find held-out MAE is flat in σ for $\sigma \geq 0.5\,\text{med}$, so a fixed median bandwidth is both simpler and more robust. One response-kernel pass produces the entire σ-grid, so tuning is nearly free. We use a single regressor (distance-weighted k-NN) and a logit-space, bias-decomposed head throughout.

Estimation pipeline. For each replicate we (i) reduce response and query embeddings by PCA on the *observed* rows only (dimension by the second Zhu–Ghodsi elbow); (ii) form D from Eq. 2; (iii) embed models by classical MDS; (iv) map the embedding to scores with the *same* head as the matrix-completion baseline up to the factor model—logit transform, per-task standardization to unit variance, and a two-way (model + task) additive bias—then a k-NN regression of the residual on the embedding (replacing completion's low-rank factor), predicting every cell (held-out cells directly, observed cells leave-one-out so the estimate never sees its own target). Matching the per-task standardization is important: without it the model offset is dominated by the highest-logit-variance tasks, which silently handicaps the embedding on heterogeneous, multi-scale suites.

The embedding-⊕-score ensemble. For each cell we combine the embedding with whichever score signal carries independent information. On a *missing* cell, that is matrix completion of the (noisy) score matrix [Papailiopoulos et al., 2026]; we blend it with PKPS using a weight fit by held-out cross-validation against the observed scores. On an *observed* cell, the cell's own sample mean is the score signal (matrix completion merely echoes it there); we shrink it toward the PKPS estimate by a precision weight $\sigma_q^2/(\sigma_q^2 + e_{PKPS}^2)$—the sample's noise variance over its total disagreement with PKPS—so the own sample dominates as queries grow and the embedding dominates as noise

grows. Both quantities are read off the observed queries (the noise variance from a half-split of each cell's queries), so the score baselines see exactly the same information.

3.3 Query efficiency

PKPS is designed to reach a target prediction accuracy from fewer evaluated queries than paired DKPS: by bridging responses to similar queries it uses evidence that DKPS, limited to shared queries, discards. We state the expected gain as a conjecture and leave a formal analysis—in the style of the DKPS query-efficiency bound [Anonymous, 2026]—to future work; Section 4.2 is its empirical counterpart.

Conjecture 1 (Query efficiency of PKPS)**.** *Fix a benchmark score function that is Lipschitz on the perspective space and a target error ε. For a query design $\mathbf{Q} = (Q_1, \ldots, Q_n)$ with cross-model overlap ρ (the fraction of query-kernel mass shared across models), the number of queries per model that PKPS needs to reach error ε is at most that of paired DKPS, and the gap is increasing as $\rho \to 0$; at full overlap $\rho = 1$ the two coincide.*

4 Experiments

A real evaluation matrix is governed by three observation levers, which structure our experiments: (i) *task coverage*—the fraction of (model, task) cells run at all; (ii) *queries per cell*—how many queries a run uses, which sets the per-cell measurement *noise*; and (iii) *cohort size*—the number of models. The all-tasks-covered limit is efficient benchmarking (denoising); the zero-query limit is benchmark completion; the realistic case is a noisy mixture, and we estimate the whole matrix at once.

4.1 Synthetic data

We generate models with latent ability vectors v_i and tasks with latent vectors t_k, scores $v_i \cdot t_k + \varepsilon$, and queries drawn near each task center. Two-level missingness (task-level and query-level) mirrors real benchmarks. Figure 2 sweeps, one at a time, the number of models, number of tasks, task observation probability, query observation probability, task spread, and response noise. The *combined* estimator (all queries) consistently beats the *paired* (shared-task only) and *unpaired* (unique-task only) estimators, with the largest margin in the realistic regime: many models, sparse task overlap, moderate task spread. The advantage shrinks when overlap is abundant (paired already has the data it needs) or tasks are very dissimilar (the query kernel cannot bridge them).

4.2 Real data

We cast benchmark estimation as one problem: recover the full (model $\times$ task) score matrix from observations in which each cell is run on $m \geq 0$ queries—m=0 is a missing task, $m > 0$ a noisy sample mean. Our predictor ensembles three item-level signals where each carries independent information (Section 3): a cell's *own sample* (where $m > 0$), *matrix completion* of the score matrix (which adds information only on missing cells—on observed cells it echoes the sample), and the PKPS *embedding* (everywhere). Using HELM-MATH as a running example, we isolate *when* the embedding adds value over the score channels.

Effective rank sets the margin. Given the same preprocessing (Section 3), how much the embedding adds over completion is governed by how much structure is left for a low-rank factor to miss.

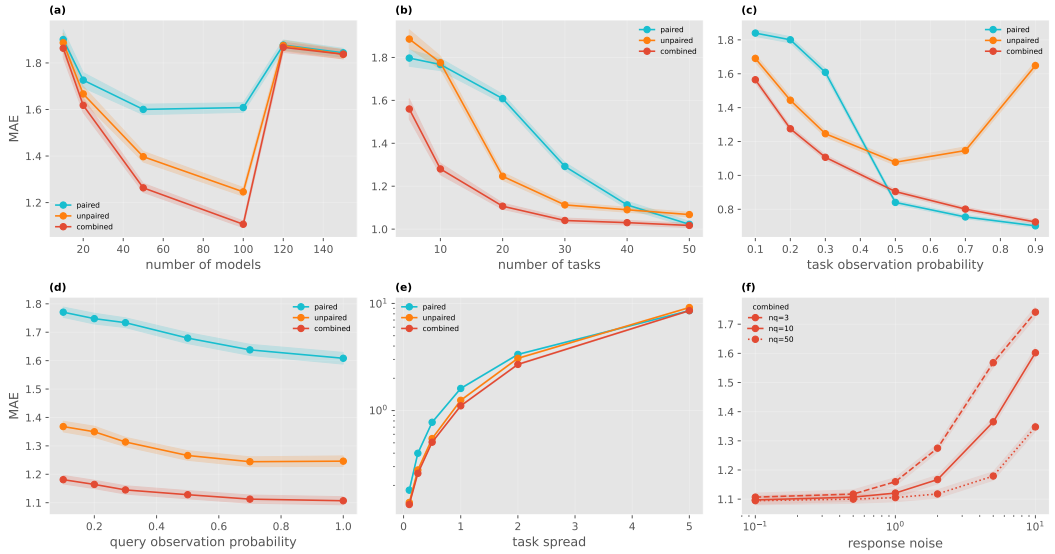

Figure 2: Synthetic study (held-out score MAE, 50 seeds). The combined product kernel dominates the paired/unpaired estimators across the number of models, number of tasks, task observation probability, query observation probability, task spread, and response noise $\times$ queries-per-task.

When the bias-removed score matrix is itself low-rank, a rank-2 factor captures it and the embedding can only match; when it is high-rank, the embedding reads structure completion cannot. The effective rank (participation ratio of the singular values) makes this concrete: MATH 4.88, the suite 3.55, WMT 1.68. On high-rank MATH PKPS beats matrix completion *at every* completion rank 2–8 (Figure 3, 0.087 vs. 0.099–0.10); on near-rank-one WMT and on the suite it *ties* completion at every rank (0.039 vs. 0.039; 0.110 vs. 0.102–0.106). So effective rank sets the *size* of the embedding's advantage—decisive when there is high-rank structure, neutral when there is not—and the embedding is never the worse choice. (Reading the suite's structure is also limited by *congruence*: half its tasks are single-token answers whose embeddings are degenerate, so the block-diagonal kernel keeps them from contaminating the rich-response blocks rather than expecting them to help.)

Noise, coverage, and cohort. Given a high-rank, congruent matrix, the embedding ensemble beats the best score-only predictor (own sample where observed, completion where missing) across all three observation levers on MATH (Figure 4), and helps most where evaluation is hardest: as the queries per cell shrink (the sample mean grows noisy while response *style* stays stable)—all the way to q=0, where the cell is simply completed, so efficient benchmarking and completion are the two ends of a single axis—as coverage drops, and —most sharply—as the cohort shrinks, where rank-2 completion collapses for lack of rows (0.22 at n=10) while the embedding, needing no rows, holds the ensemble flat. At full queries and large cohorts the score channel is already near-exact and the ensemble simply matches it.

Unpaired queries. The embedding is also what makes the method robust to *unpaired* evaluation. When models within a task answer different queries, the two competitors that exploit item structure—DKPS (identity query kernel) and IRT—lose their shared block and collapse, while the RBF query kernel still bridges similar queries (Figure 5): at a fixed 16-query budget, sliding from fully paired to fully unpaired, IRT and DKPS degrade from $\sim$0.08 to 0.28/0.24 while PKPS holds near 0.14. At very small query budgets the embedding likewise beats the sample mean and IRT by a wide margin.

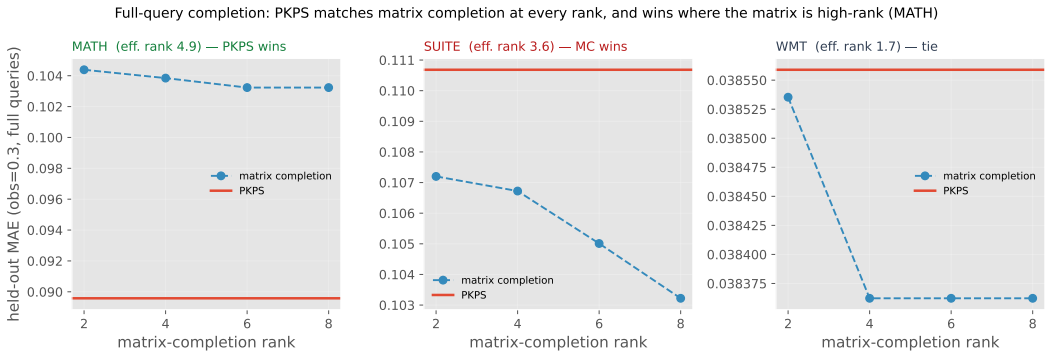

Figure 3: Full-query completion (obs. prob. 0.3). Matrix completion at ranks 2–8 (purple) vs. PKPS (blue line), with matched preprocessing. On high-rank MATH raising the completion rank never reaches PKPS; on low-rank WMT and the suite the two coincide.

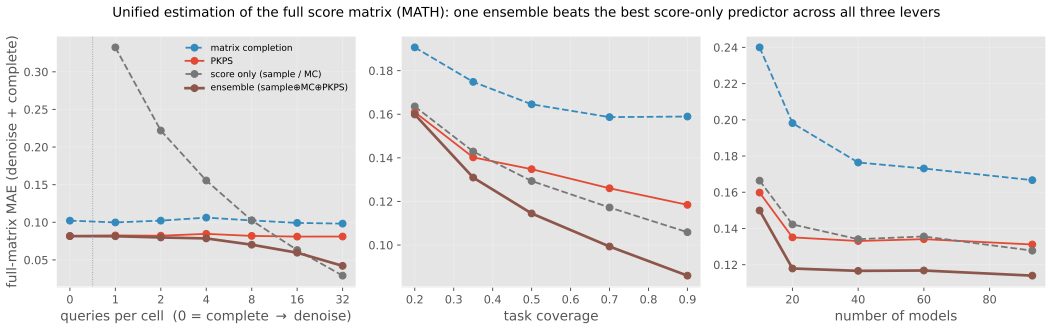

Figure 4: Unified full-matrix estimation on MATH (MAE over every cell vs. the true full score). The one sample $\oplus$ completion $\oplus$ embedding ensemble (green) beats the best score-only predictor (grey) and each channel alone across queries per cell (left; q=0 is pure completion, growing q is denoising—the two are one axis), task coverage, and cohort size. The score-only curve spikes at q=1 (a single noisy query) while the ensemble flows smoothly from the completion point downward.

The full heterogeneous suite. We now run the unified estimator on one realistic, heterogeneous benchmark: 18 tasks across four datasets with very different response types—MATH (7 math subjects), WMT-14 (5 translation directions), MedQA (medical multiple-choice), and LegalBench (5 legal classification subsets)—on the 93 models evaluated on all four. MATH and WMT have rich free-text responses; MedQA and LegalBench responses are single tokens for which text embeddings are degenerate, so we encode each dataset's responses natively (Gemini embeddings vs. one-hot answers) in disjoint blocks. With a linear k_R this makes cross-dataset response similarity exactly zero, and a single shared query embedding with a within-domain bandwidth keeps k_Q near zero across domains: the product kernel factorizes by domain, never comparing a proof to a legal label.

Even though the suite stacks every adverse condition—low aggregate rank, responses incongruent across four domains, four different scoring scales—the embedding holds up. On this matrix the PKPS channel *alone* already beats matrix completion across task coverage and cohort size, decisively so when models are scarce (0.15 vs. 0.19 at n=10, where rank-2 completion fails for lack of rows while the embedding needs none). The single sample $\oplus$ completion $\oplus$ embedding ensemble is then best across all three levers (Figure 6), beating the strongest score-only predictor by a margin that widens as evaluation gets sparser, noisier, or smaller-cohort (from a few percent at full coverage to $\sim$20% at one query per cell). We tune all weights only against the observed noisy scores, so the score baselines get exactly the same information. The earlier impression that completion dominates heterogeneous, low-rank matrices was an artifact of an unequal head: with matched preprocess-

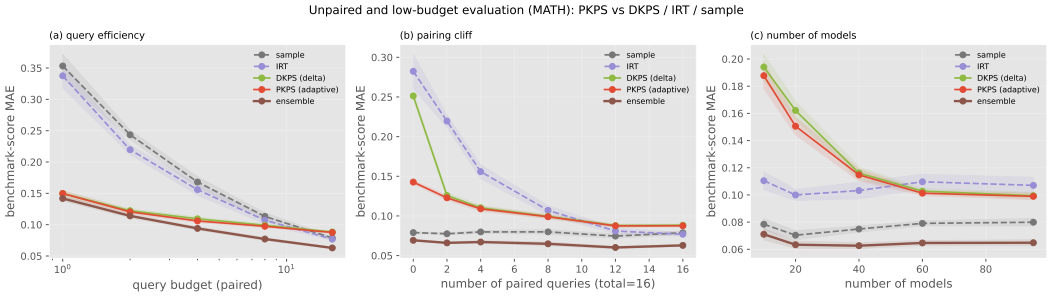

Figure 5: The embedding under unpaired and low-budget evaluation (HELM-MATH, full-score MAE). (a) At a tiny query budget DKPS/PKPS beat the sample mean and IRT. (b) Sliding from paired to unpaired at a fixed budget, IRT and DKPS collapse while PKPS survives. (c) Cohort size. The embedding-$\oplus$-sample ensemble is best throughout.

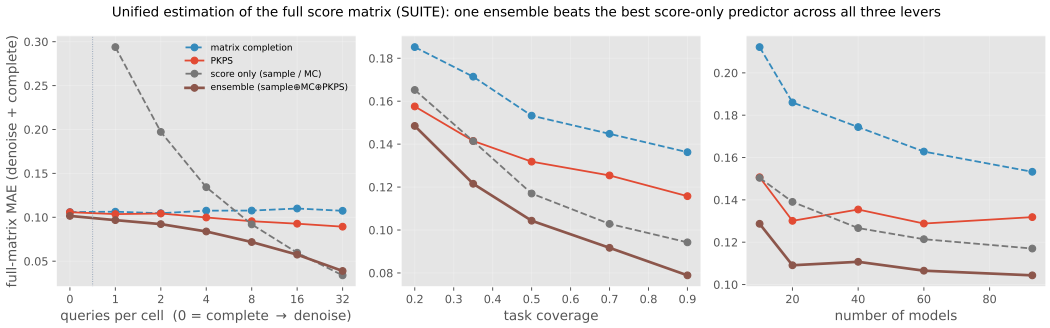

Figure 6: Unified full-matrix estimation on the heterogeneous 18-task suite (MAE over every cell vs. the true full score). The sample $\oplus$ completion $\oplus$ embedding ensemble (green) beats the best score-only predictor (grey) across queries per cell (q=0 completion through denoising—one axis), task coverage, and cohort size; at n=10 matrix completion collapses while the embedding holds the ensemble flat.

ing the item-level embedding is competitive everywhere and decisive wherever real evaluation is incomplete.

5 Discussion

Real benchmark coverage is doubly sparse and, crucially, *noisy*: each measured cell is an average over a handful of queries. We argued that the cheap way to evaluate better is to use item-level information—a cell's responses, not only its aggregate score. PKPS supplies that item-level signal as an embedding, through a product kernel that generalizes DKPS (its $\sigma \to 0$ limit) and scales to a heterogeneous 18-task suite via a block-diagonal kernel. Given the same preprocessing as low-rank completion, the embedding *matches* it on low-rank matrices and *beats* it on high-rank ones, so effective rank sets the size of its advantage; ensembled with the available item-level score—a cell's own sample where it was measured, completion where it was not—it recovers the full score matrix and beats the best score-only predictor by a margin that grows as evaluation gets sparser, noisier, or smaller-cohort. The embedding and score signals are complementary throughout, and the item-level embedding is never the worse choice—decisive wherever real evaluation is incomplete. This makes score prediction practical for organically-assembled leaderboards, where neither pairing nor exhaustive per-cell evaluation can be assumed.

References

Anonymous. Query-efficient model evaluation using cached responses. *arXiv preprint arXiv:2605.07096*, 2026.

Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *Journal of Machine Learning Research*, 2010.

Dimitris Papailiopoulos et al. You don't need to run every eval: Low-rank completion of benchmark matrices. *arXiv preprint arXiv:2606.24020*, 2026.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: Evaluating llms with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024.

Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.

Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *EACL*, 2024.